

Design File Storage:

What are popular file storages?

1. Managed services
 - a. AWS Simple Storage Service (S3)
 - b. Azure Blob
 - c. Google cloud storage
2. Open source
 - a. Hadoop Distributed File System (HDFS)
 - b. Git large file storage

We used to store data in database

Can we store very large files in the database directly?

No

Why do we need a new database or file storage for storing very large files?

Existing databases (both sql and no-sql) are poor choice for storing very large files

That's why we come up with file storage to store very large files

Use cases of large file storage:

1. Store multimedia files (images, videos) consumed by users
 - a. Youtube / Netflix → video/audio files
 - b. Scribd → pdfs
 - c. Social media → audio/video

2. Media files uploaded by users
 - a. Social media
 - b. Youtube
 - c. Stack overflow, Reddit, Quora

3. Storing logs

Example:

Google → 10Million+ servers

We have to aggregate/concatenate all server logs in a centralized location

Concatenated file size location $\geq 100\text{TB}$

Are you going to serve this much large data to the user directly?

No

We are not going to serve this much large data directly to users

We are going to use CDNs to serve this kind of large data

CDNs are actually cache

Requirement Gathering:

1. Functional requirements → features
2. Non functional requirements → design trade-offs/choices

Functional requirements:

1. client/user should be able to upload and download the files
2. Able to store very large files ($\leq 100\text{ TB}$)

3. Able to append to the existing files
4. Durability
5. Basic analytics should be possible for the client/user
 - I.e. ability to process large files
 - Count number of words
 - Find occurrences of a particular word

Non Functional requirements:

1. Latency should be low
 - You never going to serve large files like multi-media files from the DB, always serve via CDN
2. Full text search
3. Paginated queries (by time)
4. CAP
 - Consistency
 - Availability
 - Partition tolerance
 - Simultaneously only 2 out of 3 properties is satisfied
 - We want all our systems to be partition tolerant
 - So we have to choose between consistency and availability
 - Scenario-1:
 - We just uploaded a file, when they try to view that file and if the file is not showed to the user
 - Scenario-2:
 - We edited a file, but when user tries to see it, still old data is visible

Both these scenarios provide bad experience to the user

So we prefer immediate consistency

This system is CP system

Design:

Scenario:

Suppose I want to upload a 10TB file, how can I do that?

1. We can upload the complete file in a single machine or server

Pros:

We need not maintain multiple entries for each chunk

Cons:

File size is limited by server size

2. We can distribute this 10 TB across multiple servers

Divide this file into multiple parts

Each part is called chunk

We store chunks in multiple servers

Pros:

File size is not limited by server size

Cons:

We have to collate all the chunks of the file when downloading

Which option is better among both?

Since we are working with very large files, we don't want file size to be limited by server size

So we are going to divide the file into multiple chunks and those chunks in different servers

100 TB file → chunk1 chunk2 chunk3 chunk4 ... chunkN

How to keep track of all these chunks?

We will maintain metadata about the chunks

File0 → chunk0 chunk1 chunk2 chunk3

(file_id, chunk_id, server_id)

File_id → tells to which chunk the current chunk belongs to

Chunk_id → identifier of a particular chunk

server_id → tells the server in which this particular chunk stored

What is the space taken for storing metadata of a chunk?

file_id → 2b

chunk_id → 2b

server_id → 100 b

=> approximately we required some 100 bytes of data for storing metadata of a chunk

What should be the perfect size for a chunk?

1. Chunk size is too large (10GB)

uploading/downloading/processing the chunk will be difficult

So chunk size should not be too large

2. Chunk size is too small (1KB)

Number of chunks = file size / chunk size =

$100\text{TB}/1\text{KB} = 10^{11} = 100 \text{ Billion chunks}$

Meta data size of each chunk = 100 B

Meta data size for all the chunks = $100 \text{ B} * 10^{11} = 10^{13} \text{ B} = 10 \text{ TB}$

We have 10TB overhead for storing a 100TB file

This is very very bad

When chunk size is too small, then there is too much overhead of metadata

Chunk size should not be too much large or should not be too much small

HDFS 1.0 → chunk size is 64MB

HDFS 2.0 → chunk size is 128MB

If chunk size = 128 MB

Number of chunks for 100TB file = $100 * 10^{12} / 128 * 10^6 = 10^6$ chunks

Overhead for 10^6 chunks:

$10^6 * 100\text{B} = 100\text{MB}$

Which is fine

Fault tolerance:

Replicate chunks across multiple servers

Replication factor = 3

I am going to store metadata of every chunk across 3 servers

file0 chunk0 $\rightarrow$ s1, s2, s3

Can I store metadata of each chunk in any 3 random servers?

Suppose if those 3 replicas/copies of metadata of a chunk goes to one single server

What if that single server goes down, then we are going to lose all the replicas as they go to the same server

So all the replicas should not go the same server

To decide where these copies of metadata of each should go to,

We have an algorithm rack aware algorithm

Rack aware algorithm:

Server level:

All replicas should not go to the same server

i.e atleast one replica should go to different server

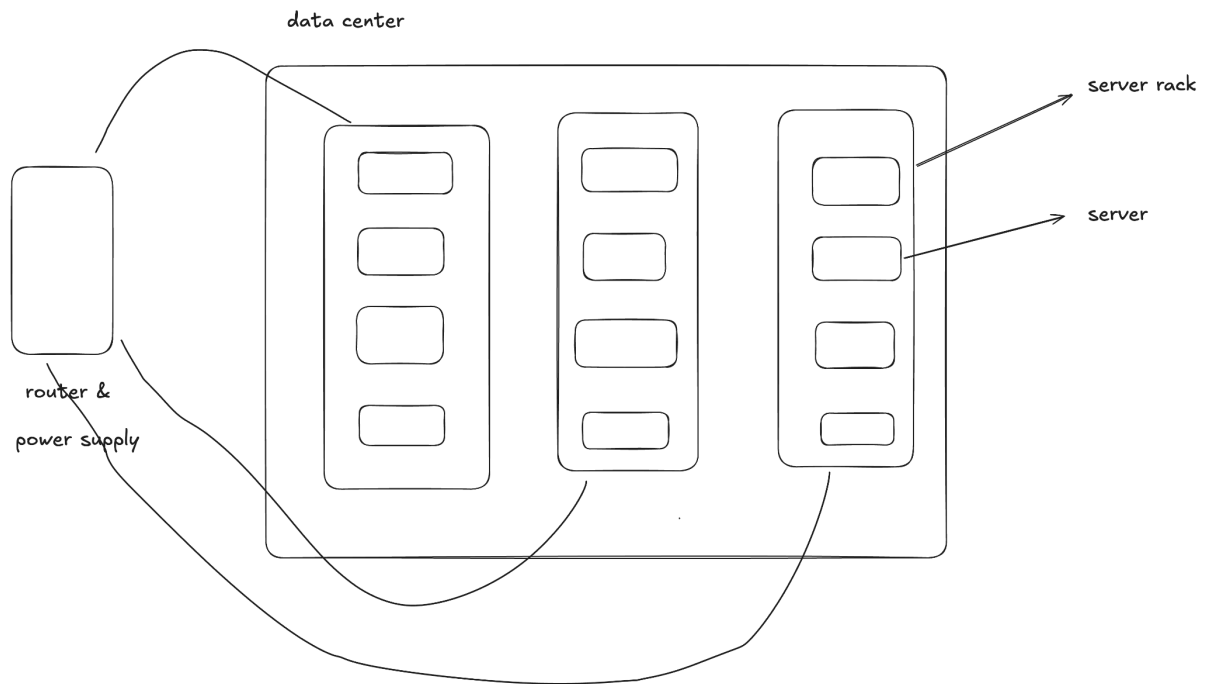
So even if one server goes down, then at least we have replica from other server

Rack level:

All replicas should not go to same server rack

Rack $\rightarrow$ In a data centre, there are lot of server racks and each rack contains server

Because suppose if we have all replicas in different servers within the same rack and power to that rack goes down, then all replicas will be lost
So at least one replica should be present in different server rack



Data center level:

All replicas should not go to the same data centre as well

I.e. at least one replica should go to the different data centre

Because what if something happens to that data centre, you are going to lose all the replicas if you keep all replicas within the single data centre

Don't keep all replicas in single data centre

Don't keep all replicas in single server rack
Don't keep all replicas in single server

File storage architecture:

User wants to upload/download very large file

Client → backend server

Since Backend server cannot store very large file, it will ask file storage to store very large file

This backend server acts as a client

File storage system acts as a backend

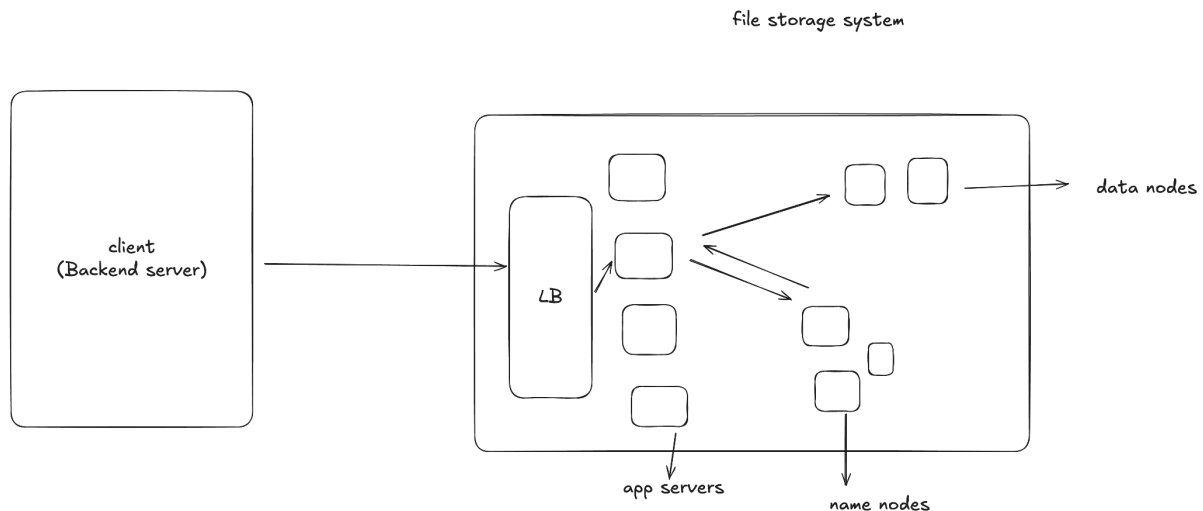
File storage system gets very large file as input

It will have some app servers → which will divide the large file into chunks

Chunk → will have some metadata

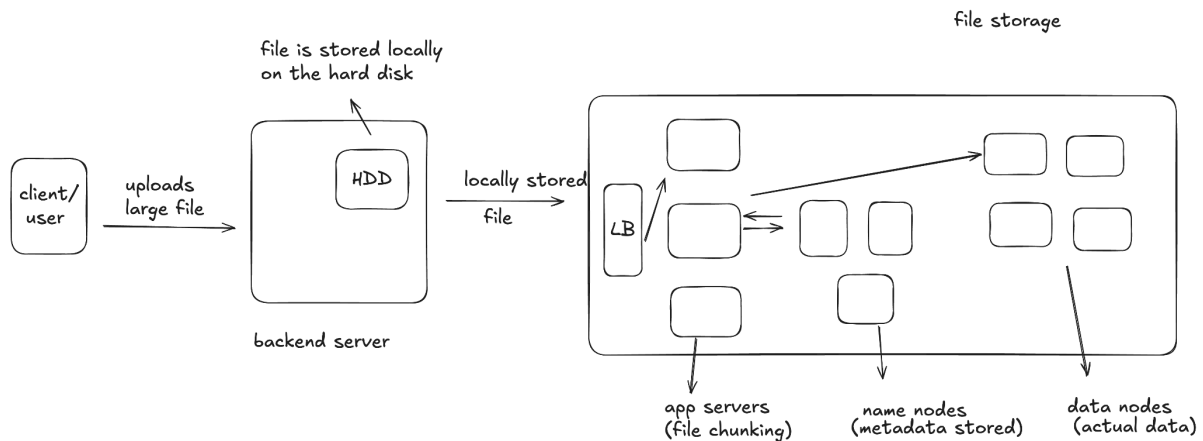
To store metadata, we are going to use some nodes called name nodes

To store actual data, we are going to have some nodes called data nodes



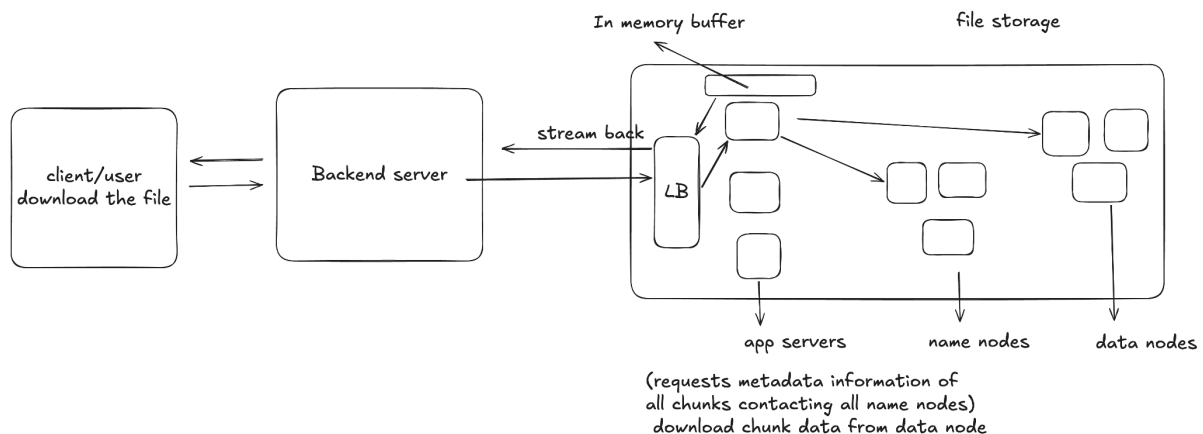
1. client(some backend server) sends a very large file
2. App servers will do the file chunking
3. Name nodes:
 - a. Servers which store metadata of file chunks → file_id, chunk_id, server_id
 - b. Name nodes are replicated using master slave architecture
(among all the replicas, one replica acts as a master and other replicas act as slaves. If master goes down, one of the slaves act as master)
4. Data nodes
 - Stores actual chunk data
 - Each chunk is replicated across multiple data nodes using rack aware algorithm

Upload flow:



1. User will upload file to the backend server
2. Backend server stores that file in its local hard disk
3. Backend server uploads that file from its local hard disk drive to file storage app server
4. Now file storage app server do the file chunking
5. Metadata of file chunks are stored in name nodes
6. File chunk data is stored in data node

Download flow:



1. Download request comes to app servers inside file storage
2. App server contacts the name nodes and asks all metadata information for all chunks of this file

- a. Exactly how many chunks, what chunks
 - b. For each chunk → what replicas and where
- 3. App server starts downloading the chunk
 - a. Try from any replica
 - b. If one replica is down, it will read from different replica
- 4. It will store this data in in-memory
- 5. It will start streaming this data down to the client
- 6. In parallel it will start downloading other chunk